WTI-crudeoil-forecasting-ARMA-VAR-R

Aniruddha Burman

2025-04-25

Load the data into R (or stata). Graph it. Does it appear to be stationary, trend stationary or stochastic?

We start the assignment by loading the data into R using the read_csv() function from the tidyverse library. To inspect the dataset and to get an idea of what we are working with, we used the glimpse() function, from the dplyr package, which is essentially a transposed version of the print() function, where columns run down the page and the data runs across. Although we would be working with only one variable from the dataset, it is useful to know what the entire dataset is like. We identify that our dataset has 178 observations and 11 variables. The dates in the dataset begin from 1st January, 1980. We also notice that the variable assigned to us, "DCOILWTICO" has quite a few missing values in the early parts of the dataset.

We then proceed to create a time variable in the dataset, "t" which numbers the observations from 1 to 178, using the mutate() function. After creating the time variable, we convert the dataset using the as_tsibble() function. This allows us to handle time series data better in R. Then we extract the "DCOILWTICO" variable from the dataset using the select() function, and simultaneously removed the NA values from the variable using drop_na() function. We plot the data using a simple ggplot() function from the ggplot2 library. We kept the t variable on the x-axis and the Crude Oil Prices in the y-axis. Looking at the plot we can confirm that the data is not stationary, owing to all the large swings throughout the plot. It appears to be following a stochastic trend, likely affected by random shocks.

```r
library(haven)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(tsibble)
```

```
## Registered S3 method overwritten by 'tsibble':
##   method               from
##   as_tibble.grouped_df dplyr
##
## Attaching package: 'tsibble'
##
## The following object is masked from 'package:lubridate':
##
##     interval
```

```
##
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, union
```

```r
library(dplyr)
```

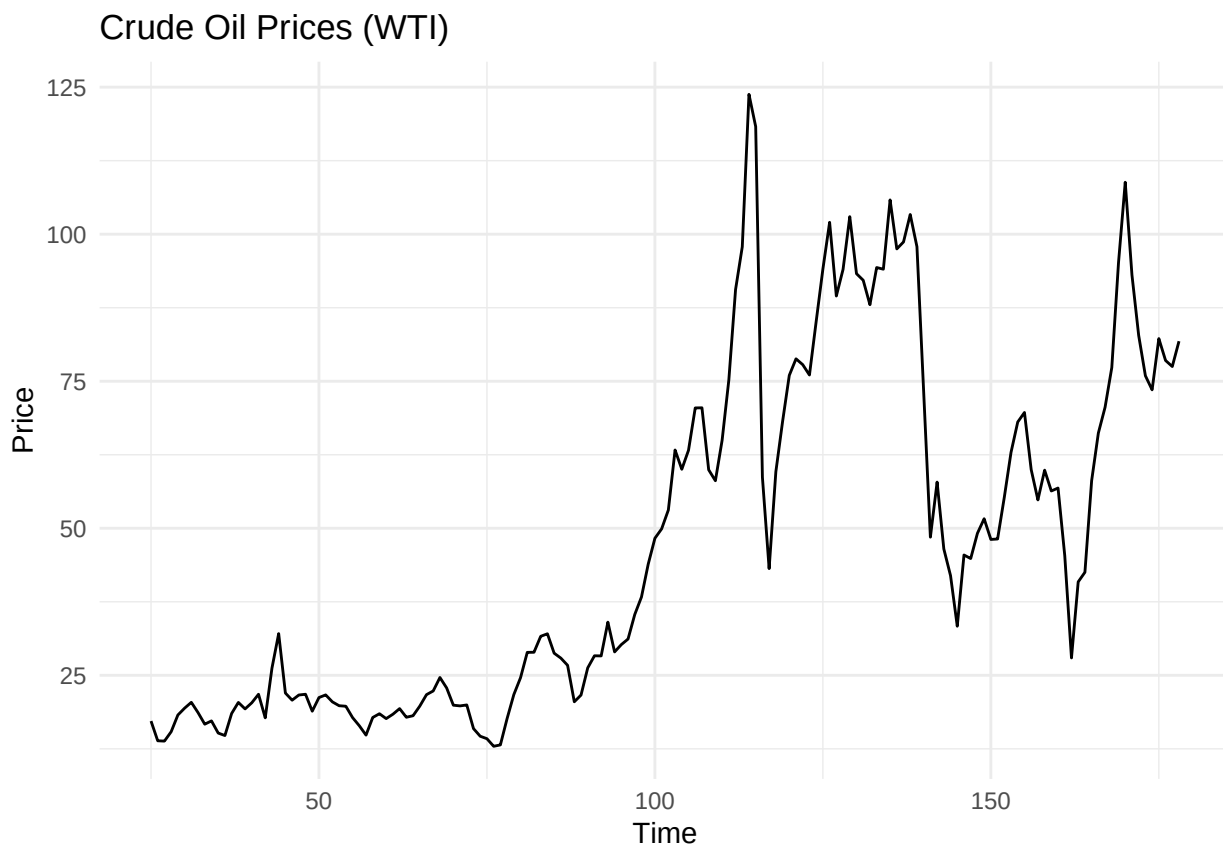
```r
df = read_csv("assignment.csv")
```

```
## Rows: 178 Columns: 11
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl  (10): GDPC1, RSXFS, PPIACO, PCECTPI, DCOILWTICO, USSTHPI, WPU02540105, ...
## date  (1): datestr
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
glimpse(df)
```

```
## Rows: 178
## Columns: 11
## $ datestr       <date> 1980-01-01, 1980-04-01, 1980-07-01, 1980-10-01, 1981-01~
## $ GDPC1         <dbl> 7341.557, 7190.289, 7181.743, 7315.677, 7459.022, 7403.7~
## $ RSXFS         <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ PPIACO        <dbl> 86.533, 88.267, 91.167, 93.267, 96.100, 98.267, 98.933, ~
## $ PCECTPI       <dbl> 37.545, 38.464, 39.364, 40.338, 41.384, 42.078, 42.772, ~
## $ DCOILWTICO    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ USSTHPI       <dbl> 100.00, 101.66, 104.46, 104.57, 105.19, 107.41, 108.85, ~
## $ WPU02540105   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ COMPUTSA      <dbl> 1766, 1588, 1393, 1321, 1347, 1355, 1260, 1136, 984, 990~
## $ NGDPRSAXDCCAQ <dbl> 223045.8, 222948.3, 222763.0, 225845.3, 230710.8, 233233~
## $ PCEC96        <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
```

```r
df = df %>% mutate(t = 1:nrow(df))
df = df %>% as_tsibble(index = t)
crude = df %>% dplyr::select(DCOILWTICO) %>% drop_na()
crude %>%
  ggplot(aes(x = t, y = DCOILWTICO)) +
  geom_line() +
  labs(title="Crude Oil Prices (WTI)", x="Time", y="Price") +
  theme_minimal()
```



Formally test for the presence of a unit root. Explain what version of the test you used and why you used it.

In the second part of this question, we conduct a formal test for stationarity after suspecting that the series may be stochastic. We use the Augmented Dickey-Fuller (ADF) test, to check whether our time series contains a unit root. We use the ADF test with a trend because our graph showed no constant mean, but instead a drifting behavior, a possible presence of a deterministic trend. We use the ur.df() function from the urca package to carry out the test, and specified the test regression with a trend term using the "type=trend" command within the ur.df() function. From the results of the ADF test we observe that the test statistic is -3.5425, which is lower than the 5% critical value of the test statistic, -3.43. Hence, we reject the null hypothesis of a unit root. Thus, although from visual inspection we thought that crude oil prices were stochastic, the ADF test helped us identify that the crude oil prices are trend stationary.

```
library(urca)
test_adf = ur.df(crude$DCOILWTICO, type="trend", selectlags = "AIC")
summary(test_adf)
```

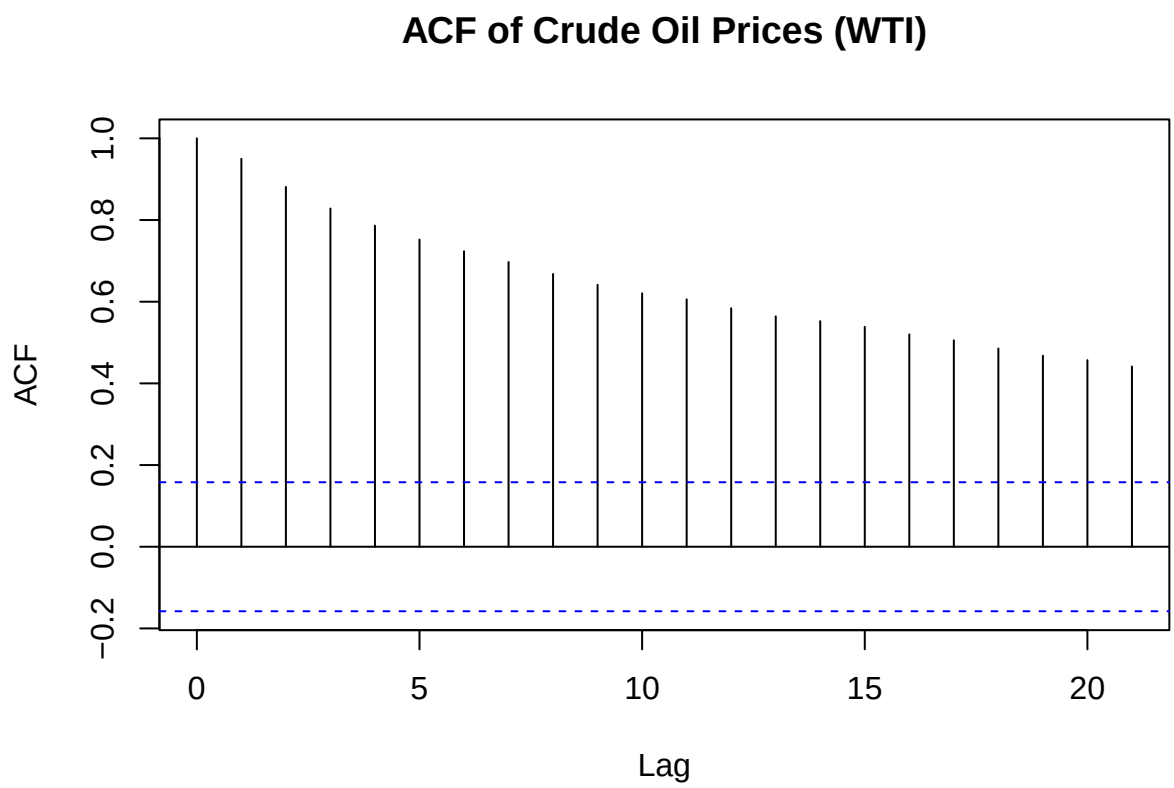
```
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression trend
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + tt + z.diff.lag)
```

```
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -50.582  -2.631  -0.030   2.880  28.958
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.41591    1.37274   1.031 0.304015
## z.lag.1     -0.12132    0.03425  -3.542 0.000531 ***
## tt           0.06009    0.02264   2.654 0.008812 **
## z.diff.lag   0.28440    0.07880   3.609 0.000420 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 8.133 on 148 degrees of freedom
## Multiple R-squared:  0.1243, Adjusted R-squared:  0.1066
## F-statistic: 7.004 on 3 and 148 DF,  p-value: 0.0001946
##
##
## Value of test-statistic is: -3.5425 4.2812 6.2747
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau3 -3.99 -3.43 -3.13
## phi2  6.22  4.75  4.07
## phi3  8.43  6.49  5.47
```

Examine the autocorrelation function and propose several possible models that may have generated the series.

Once we established using the ADF test that crude oil prices are trend stationary, we examine the autocorrelation structure of the series in this part of the question. With the help of the ACF, we can select a good model for forecasting later on in the assignment. To plot the ACF, we use the acf() function from base R. The ACF computes the correlation of the series with its own lags, helping us identify how past values influence current values. The plot obtained shows us that the autocorrelations in the plot were highly significant in the first few lags, and decayed slowly in the following lags. A characteristic of a highly persistent process is the gradual decay we see in our ACF plot, hinting at the fact that crude oil prices are influenced by long-term shocks. The pattern is inconsistent with a simple MA process since the autocorrelations are not sharply cutoff after a small number of lags. The slow decay is a characteristic of an Autoregressive (AR) process or even an Autoregressive Moving Average (ARMA) process. Hence, we propose the AR(1), AR(2) and ARMA(1,1) models as potential candidates representing the crude oil prices series.

```r
acf(crude$DCOILWTICO, main="ACF of Crude Oil Prices (WTI)")
```



Estimate the model(s) proposed in part 1c.

We estimate the previously proposed models in this part of the question, namely, AR(1), AR(2) and ARMA(1,1). We use the arima() function which allows us to fit our models. We use the stargazer package as well to organize our regression outputs into well-formatted and neatly organized tables. From the output we can tell that all 3 models produced statistically significant coefficients at the 1% level.

```
ar1_model = arima(crude$DCOILWTICO, order=c(1,0,0))
ar2_model = arima(crude$DCOILWTICO, order=c(2,0,0))
arma11_model = arima(crude$DCOILWTICO, order=c(1,0,1))
library(stargazer)
```

```
##
## Please cite as:

##  Hlavac, Marek (2022). stargazer: Well-Formatted Regression and Summary Statistics Tables.

##  R package version 5.2.3. https://CRAN.R-project.org/package=stargazer
```

```
stargazer(ar1_model, ar2_model, arma11_model, type="text")
```

```
##
## ===============================================
##                         Dependent variable:
##                     ----------------------------
##
##                       (1)       (2)       (3)
## -----------------------------------------------
```

```
## ar1               0.959***  1.199***  0.930***
##                    (0.022)   (0.078)   (0.030)
##
## ar2                         -0.251***
##                              (0.078)
##
## ma1                                    0.372***
##                                         (0.083)
##
## intercept         47.978*** 47.868*** 47.783***
##                   (14.471)  (11.642)  (11.673)
##
## ----------------------------------------------
## Observations         154       154       154
## Log Likelihood    -548.925  -543.926  -541.301
## sigma2             71.859    67.290    64.990
## Akaike Inf. Crit. 1,103.851 1,095.852 1,090.603
## ==============================================
## Note:               *p<0.1; **p<0.05; ***p<0.01
```

Assess your model(s) using if possible formal tests and/or an information criterion.

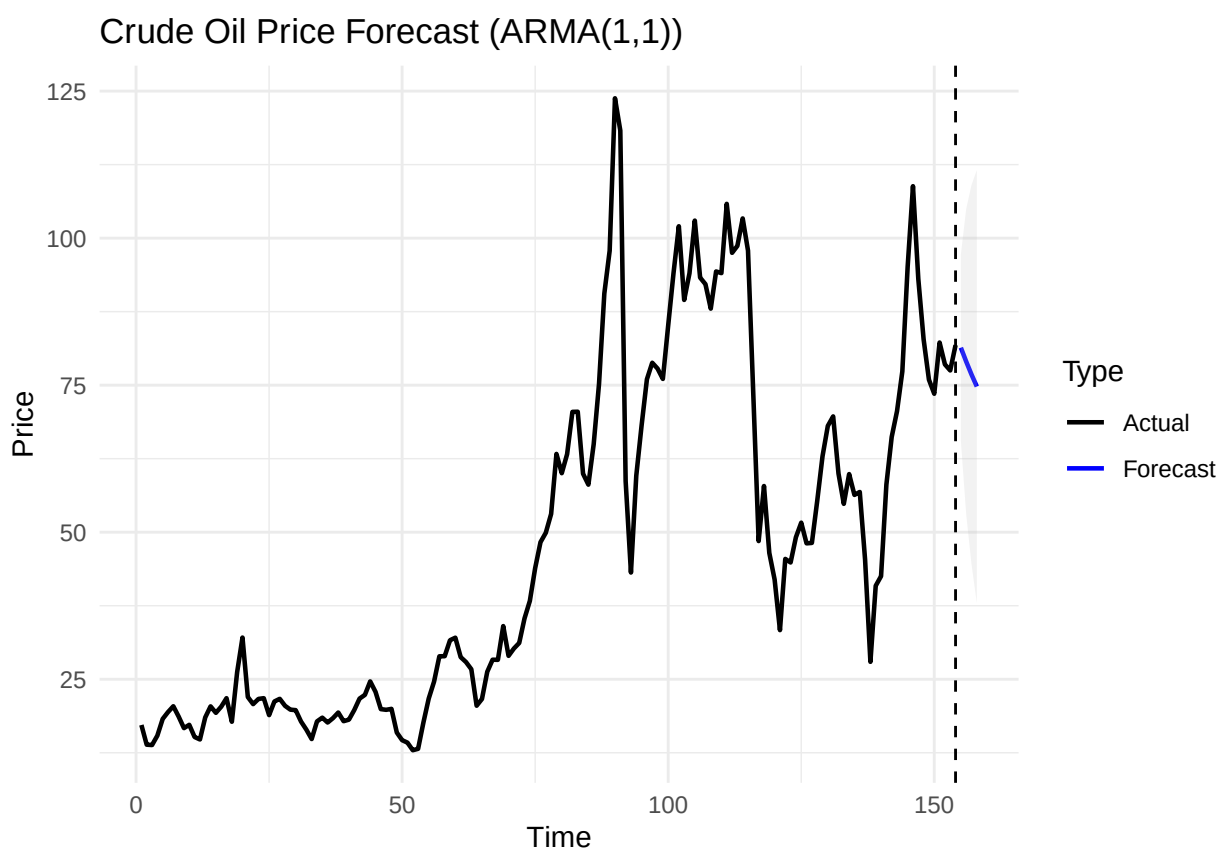
From the table in the previous part of this question, we obtained the AIC values for all 3 models which is a really good criterion for model comparison. The AIC balances the goodness of fit and model complexity helping us choose the best model. We notice that the ARMA(1,1) has the lowest AIC value, hence it is chosen as the best model. It shows us that the ARMA(1,1) model provides the best fit for the data, while keeping model complexity at an acceptable level. Additionally, the ARMA(1,1) model captures both the strong persistence in crude oil prices via the AR(1) term and the impact of short-term shocks via the MA(1) term, making it the best model for forecasting.

Use the model you deem most appropriate to generate forecasts for the next year. Plot forecasted and actual observations. Also include 95% forecast error bands.

Once we confirmed ARMA(1,1) as our most preferred model for forecasting, we generate the forecasts in this part of the question. We use the predict() function to conduct the forecasts, which produces both point forecasts and standard errors, allowing us to construct the 95% confidence intervals around the forecasts. Then we create a tidy dataframe combining the actual historical data and the forecasted values. Following which we use the ggplot() function to plot the full time series data including the historical values, the forecasted future values and the 95% confidence bands. We created a dashed line in the plot to separate the historical values from the forecasted values.

```r
forecast_arma11 = predict(arma11_model, n.ahead=4)
forecast_mean = forecast_arma11$pred
forecast_se = forecast_arma11$se
lower_bound = forecast_mean - 1.96 * forecast_se
upper_bound = forecast_mean + 1.96 * forecast_se
forecast_df = tibble(
  Time = (nrow(crude)+1):(nrow(crude)+4),
  Forecast = as.numeric(forecast_mean),
  Lower95 = as.numeric(lower_bound),
  Upper95 = as.numeric(upper_bound)
)
crude_df = crude %>% as_tibble()
crude_df = crude_df %>% mutate(Time = row_number())
actual_df = crude_df %>%
```

```r
  dplyr::select(Time, DCOILWTICO) %>%
  rename(Value = DCOILWTICO) %>%
  mutate(Type = "Actual")
forecast_df = forecast_df %>%
  rename(Value = Forecast) %>%
  mutate(Type = "Forecast")
full_df = bind_rows(actual_df, forecast_df)
full_df %>%
  ggplot(aes(x=Time, y=Value, color=Type)) +
  geom_line(linewidth=0.8) +  # Thicker lines for clarity
  geom_ribbon(data=forecast_df, aes(x=Time, ymin=Lower95, ymax=Upper95),
              inherit.aes = FALSE, alpha=0.2, fill="grey") +
  geom_vline(xintercept=nrow(crude_df), linetype="dashed", color="black") +
  labs(title="Crude Oil Price Forecast (ARMA(1,1))",
       x="Time", y="Price") +
  scale_color_manual(values=c("Actual"="black", "Forecast"="blue")) +
  theme_minimal()
```



Assess the forecasting power of your model.

In the final part of this question, we calculate the Mean Squared Forecast Error (MSFE) to evaluate the accuracy of the ARMA(1,1) model's forecasts. The MSFE measures the average squared difference between the actual observed values and the forecasted values and serves as a quantitative indicator of forecast performance. We extracted the actual observed crude oil prices for periods 155 - 158 of the dataset, then we compared these actual values to the forecasted values generated by our ARMA(1,1) model. Finally, we calculated the squared differences between actual and predicted values and took the average of the squared errors. The low

MSFE value tells us that the ARMA(1,1) model provided a fairly accurate forecast for crude oil prices. Thus, we can conclude by saying that the ARMA(1,1) model is good forecasting model for the crude oil prices.

```
actual_future = crude_df %>%
  filter(Time %in% (nrow(crude_df)-3):nrow(crude_df)) %>%
  pull(DCOILWTICO)
predicted_future = forecast_df$Value
errors = actual_future - predicted_future
msfe = mean(errors^2)
print(paste("Mean Squared Forecast Error (MSFE):", round(msfe, 3)))
```

```
## [1] "Mean Squared Forecast Error (MSFE): 12.769"
```

Choose another variable and explain why it is reasonable to expect that it will improve the forecast of your assigned variable.

In the second question of our assignment, we have to extend the forecasting model by incorporating a second variable and constructing a Vector Autoregression (VAR) model. Out of all the available variables in the dataset, we chose PPIACO, which is the Producer Price Index for All Commodities, as the second variable for our VAR model. We chose this variable because crude oil is used in the production of a lot of commodities. Changes in the price of crude oil affects production costs for these commodities, which in turn influences the Producer Price Index. Thus, there is a strong justification that there is a relationship between crude oil prices and commodity prices. From a forecasting perspective, incorporating PPIACO into the model captures market movements and cost pressures that are not fully explained by crude oil prices by itself, improving the predictive power of the model.

Are the new variable(s) stationary?

We used a similar method to question 1b to check for stationarity. The ADF test on PPIACO gives us a test statistic of -2.7265, which is greater than the 5% critical value of -3.43. Hence, we fail to reject the null hypothesis of a unit root at the 5% significance level. We conclude that PPIACO is non-stationary.

```
ppi = df %>% dplyr::select(PPIACO) %>% drop_na()
adf_ppi = ur.df(ppi$PPIACO, type="trend", selectlags="AIC")
summary(adf_ppi)
```

```
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression trend
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + tt + z.diff.lag)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -23.7815  -1.1635   0.0199   1.2373  15.4774
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.11401    1.54675   2.660  0.00856 **
## z.lag.1     -0.05196    0.01906  -2.726  0.00707 **
## tt           0.04857    0.01705   2.848  0.00493 **
```

```
## z.diff.lag   0.46224    0.06836   6.761 2.04e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.508 on 172 degrees of freedom
## Multiple R-squared:  0.226,  Adjusted R-squared:  0.2125
## F-statistic: 16.74 on 3 and 172 DF,  p-value: 1.382e-09
##
##
## Value of test-statistic is: -2.7265 4.017 4.0574
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau3 -3.99 -3.43 -3.13
## phi2  6.22  4.75  4.07
## phi3  8.43  6.49  5.47
```

Decide how many lags the VAR should have. Explain your reasoning.

In this part of the question, we determine the appropriate number of lags to include into the model. This is essential because the number of lags affect the model's ability to capture dynamics, influence forecast accuracy and parameter estimation efficiency. We use the VARselect() function from the vars package to perform our lag selection for this part of the question. We set the maximum lag length at eight and included a constant term in the model. Our three criteria; AIC, HQ and BIC, identified one lag as the optimal lag length. BIC which accounts for overfitting the most out of the other 2 criteria, achieved its minimum at one lag.

```r
library(dplyr)
library(tidyr)
library(vars)
```

```
## Loading required package: MASS

##
## Attaching package: 'MASS'

## The following object is masked from 'package:dplyr':
##
##     select

## Loading required package: strucchange

## Loading required package: zoo

##
## Attaching package: 'zoo'

## The following object is masked from 'package:tsibble':
##
##     index

## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric

## Loading required package: sandwich

##
## Attaching package: 'strucchange'
```

```
## The following object is masked from 'package:stringr':
##
##     boundary
```

```
## Loading required package: lmtest
```

```
var_raw = df %>%
  dplyr::select(datestr, DCOILWTICO, PPIACO) %>%
  drop_na()
var_raw = var_raw %>%
  mutate(diff_ppi = c(NA, diff(PPIACO))) %>%
  drop_na()
var_data = var_raw %>%
  dplyr::select(DCOILWTICO, diff_ppi)
lag_selection = VARselect(var_data, lag.max = 8, type = "const")
lag_selection$criteria
```

```
##                    1             2             3             4             5
## AIC(n) -5.457653e+01 -5.397793e+01 -5.393389e+01 -5.385257e+01 -5.375877e+01
## HQ(n)  -5.447643e+01 -5.380276e+01 -5.368364e+01 -5.352724e+01 -5.335837e+01
## SC(n)  -5.433018e+01 -5.354682e+01 -5.331801e+01 -5.305193e+01 -5.277337e+01
## FPE(n)  1.984874e-24  3.612268e-24  3.776545e-24  4.099740e-24  4.508565e-24
##                    6             7             8
## AIC(n) -5.375458e+01 -5.365214e+01 -5.363928e+01
## HQ(n)  -5.327910e+01 -5.310158e+01 -5.301366e+01
## SC(n)  -5.258442e+01 -5.229721e+01 -5.209959e+01
## FPE(n)  4.535828e-24  5.037901e-24  5.120209e-24
```

Estimate the VAR. Report the coefficients.

We estimated a VAR(1) model including Crude Oil Prices and the first difference of PPIACO using the VAR() function from the vars package and specifying a constant term. The estimation results tell us that Crude Oil Prices, specifically their first lag, has a positive and highly statistically significant coefficient, indicating strong persistence in oil prices over time. The first lag of diff_ppi is positive, but it is statistically significant only by a small margin at the 10% level, which implies a weak influence of producer price inflation changes on crude oil prices in the short run. The R-squared values are 0.918 and 0.1959 which implies that the model explains a large proportion of variation in crude oil prices but very less variation in the differenced PPI series. The residuals of the two equations are strongly positively correlated having a value of 0.8458, suggesting that unobserved shocks affecting both series are likely related.

```
var_raw = df %>%
  dplyr::select(datestr, DCOILWTICO, PPIACO) %>%
  drop_na()
var_raw = var_raw %>%
  mutate(diff_ppi = c(NA, diff(PPIACO))) %>%
  drop_na()
var_data = tibble(
  DCOILWTICO = var_raw$DCOILWTICO,
  diff_ppi = var_raw$diff_ppi
)
var_model = VAR(var_data, p = 1, type = "const")
summary(var_model)
```

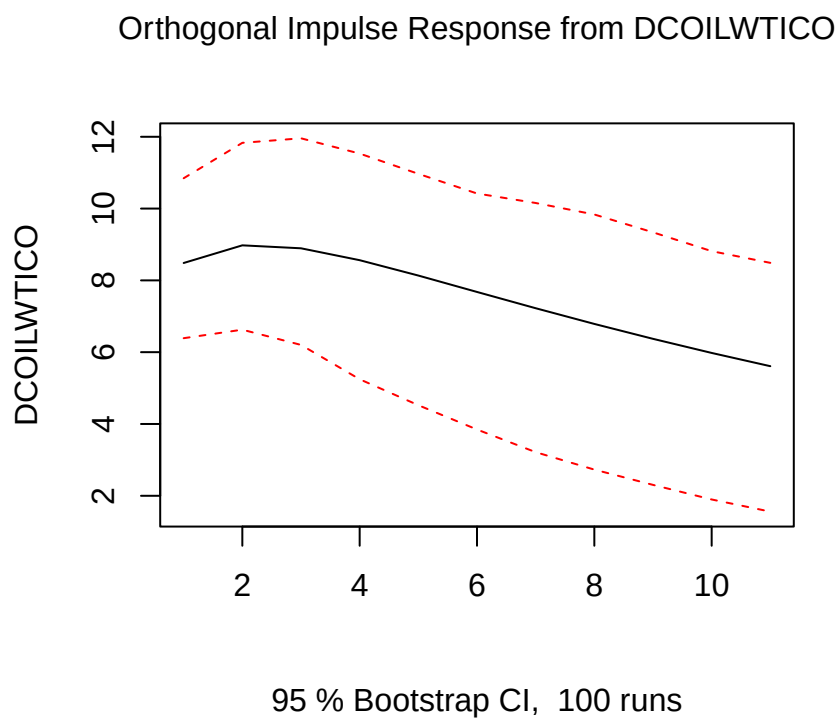
```
##
## VAR Estimation Results:
## =========================
```

```
## Endogenous variables: DCOILWTICO, diff_ppi
## Deterministic variables: const
## Sample size: 152
## Log Likelihood: -861.334
## Roots of the characteristic polynomial:
## 0.938 0.4651
## Call:
## VAR(y = var_data, p = 1, type = "const")
##
##
## Estimation results for equation DCOILWTICO:
## ===========================================
## DCOILWTICO = DCOILWTICO.l1 + diff_ppi.l1 + const
##
##               Estimate Std. Error t value Pr(>|t|)
## DCOILWTICO.l1  0.94694    0.02423  39.077   <2e-16 ***
## diff_ppi.l1    0.29192    0.16847   1.733   0.0852 .
## const          2.66498    1.31140   2.032   0.0439 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 8.484 on 149 degrees of freedom
## Multiple R-Squared: 0.918,   Adjusted R-squared: 0.9169
## F-statistic: 833.5 on 2 and 149 DF,  p-value: < 2.2e-16
##
##
## Estimation results for equation diff_ppi:
## =========================================
## diff_ppi = DCOILWTICO.l1 + diff_ppi.l1 + const
##
##               Estimate Std. Error t value Pr(>|t|)
## DCOILWTICO.l1 -0.01468    0.01090  -1.347   0.1800
## diff_ppi.l1    0.45624    0.07575   6.023 1.28e-08 ***
## const          1.26572    0.58969   2.146   0.0335 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 3.815 on 149 degrees of freedom
## Multiple R-Squared: 0.1959,  Adjusted R-squared: 0.1851
## F-statistic: 18.15 on 2 and 149 DF,  p-value: 8.83e-08
##
##
##
## Covariance matrix of residuals:
##            DCOILWTICO diff_ppi
## DCOILWTICO      71.98    27.38
## diff_ppi        27.38    14.55
##
## Correlation matrix of residuals:
##            DCOILWTICO diff_ppi
## DCOILWTICO     1.0000   0.8458
## diff_ppi       0.8458   1.0000
```

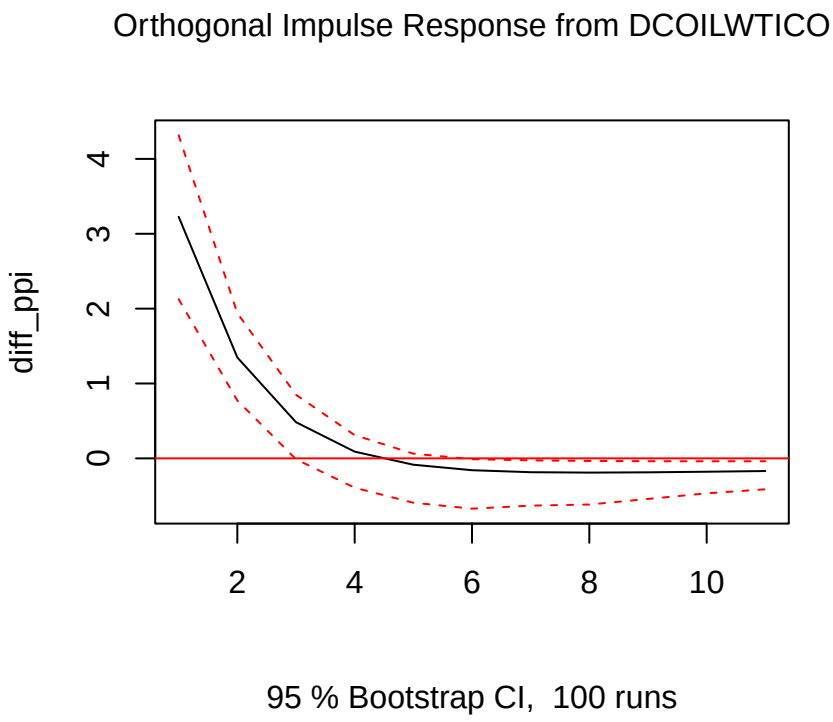
Graph the impulse functions. Comment on their realism and/or economic interpretation.

The impulse responses given by the irf() function appear realistic and consistent with economic intuition. A positive shock to the crude oil prices causes an increase in future crude oil prices, which makes complete economic sense since oil price shocks usually occur due to major supply or demand changes that do not reverse immediately. The response of producer price inflation to a crude oil price shock is positive but short-lived which again makes complete economic sense as we have noticed a change in input prices affect a commodity in the short-run but in the long-run the firms adjust prices and costs stabilize. A shock to the producer price inflation has a very insignificant effect on crude oil prices which aligns with our understanding that changes in oil prices are not caused by domestic price movements, but instead by global changes in supply and demand. Thus, our IRFs reinforce the credibility of our estimated VAR model for capturing the dynamics between crude oil prices and inflation.

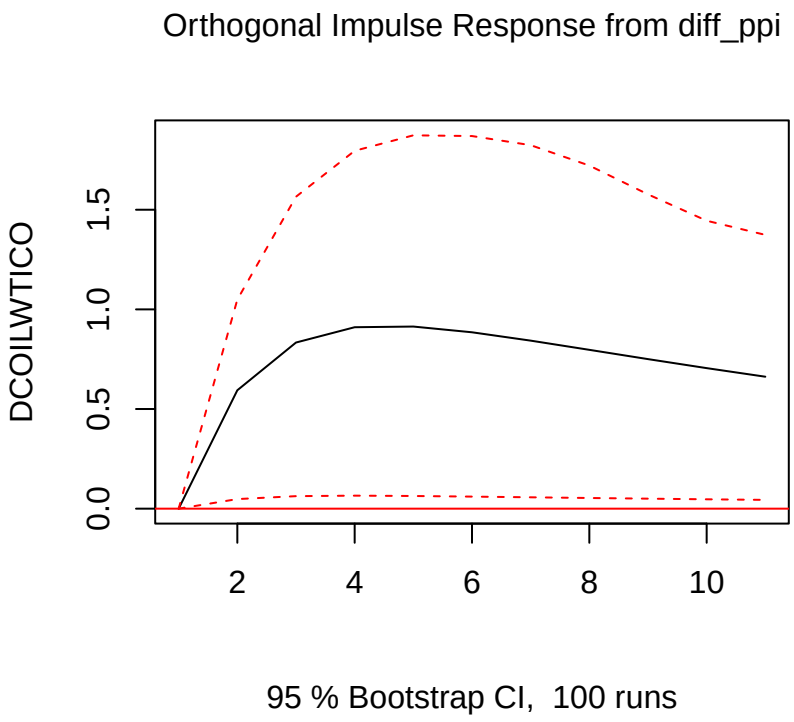
```
irf_crude_crude = irf(var_model, impulse = "DCOILWTICO",
                      response = "DCOILWTICO",n.ahead = 10, boot = TRUE)
plot(irf_crude_crude)
```



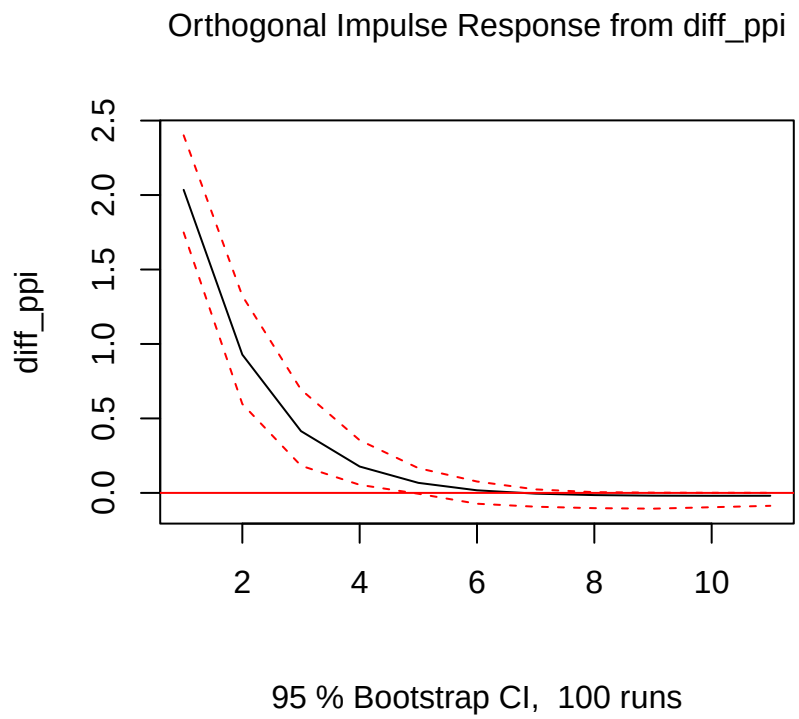

```
irf_crude_ppi = irf(var_model, impulse = "DCOILWTICO",
                    response = "diff_ppi",n.ahead = 10, boot = TRUE)
plot(irf_crude_ppi)
```




```
irf_ppi_crude = irf(var_model, impulse = "diff_ppi",
                    response = "DCOILWTICO",n.ahead = 10, boot = TRUE)
plot(irf_ppi_crude)
```




```r
irf_ppi_ppi = irf(var_model, impulse = "diff_ppi",
                  response = "diff_ppi", n.ahead = 10, boot = TRUE)
plot(irf_ppi_ppi)
```



Test the system for Granger causation. Explain the implications of your results for forecasts of your assigned variable.

We tested for the Granger causality between Crude Oil Prices and the first difference of producer price inflation using the causality() function from the vars package. The results fail to reject the null hypothesis both ways and suggest that past crude oil prices do not provide strong predictive content for changes in producer price inflation and vice-versa. Although, for diff_ppi, the result is marginally significant at the 10% level which implies that there is some limited predictive power, but still not strong enough to be definitive. The tests for instantaneous causality strongly reject the null hypothesis in both directions, which leads us to the conclusion that shocks to crude oil prices and inflation are highly correlated. We can say that while lagged inflation data may offer some marginal forecasting improvement for crude oil prices, most of the forecast accuracy still heavily relies on the autoregressive behavior of the crude oil prices themselves.

```r
granger_ppi_to_crude = causality(var_model, cause = "diff_ppi")
print(granger_ppi_to_crude)
```

```
## $Granger
##
##  Granger causality H0: diff_ppi do not Granger-cause DCOILWTICO
##
## data:  VAR object var_model
## F-Test = 3.0026, df1 = 1, df2 = 298, p-value = 0.08416
##
##
## $Instant
##
##  H0: No instantaneous causality between: diff_ppi and DCOILWTICO
```

```
##
## data:  VAR object var_model
## Chi-squared = 63.393, df = 1, p-value = 1.665e-15
```

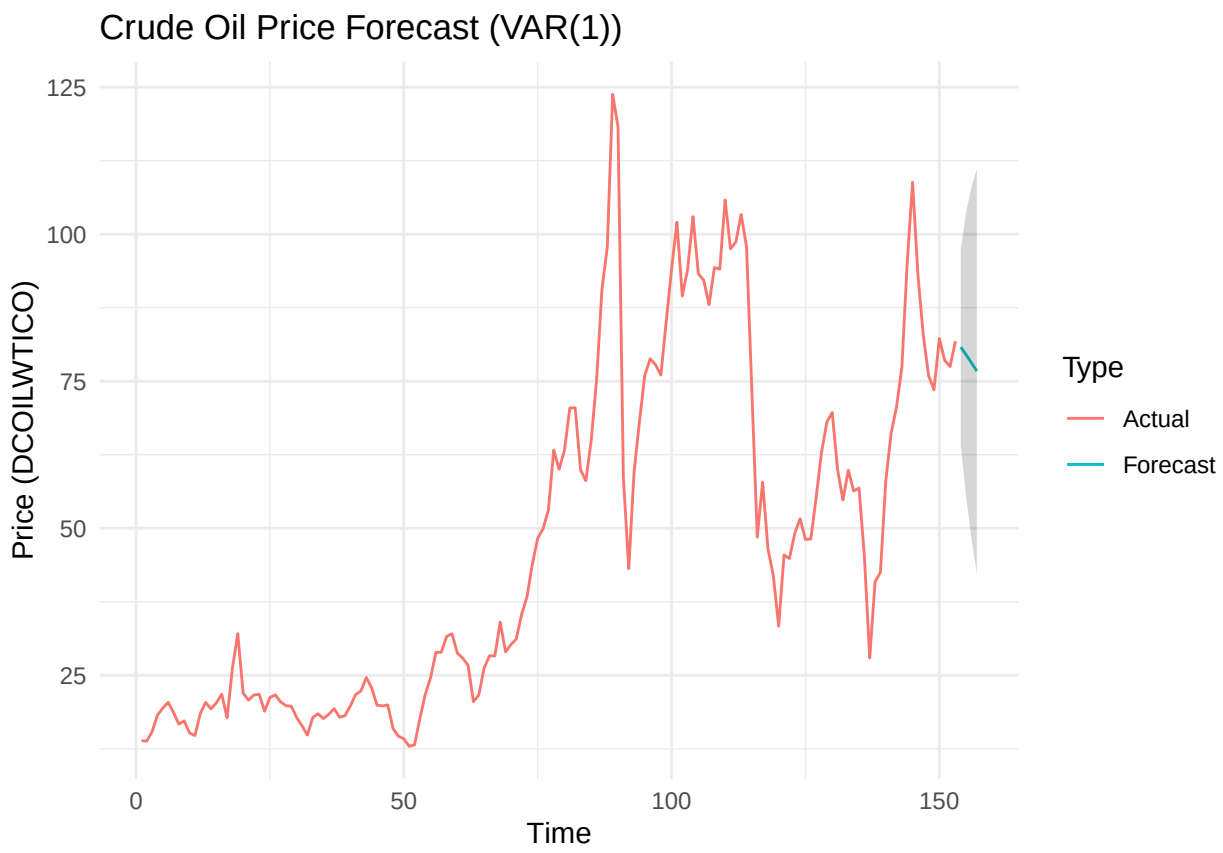
```r
granger_crude_to_ppi = causality(var_model, cause = "DCOILWTICO")
print(granger_crude_to_ppi)
```

```
## $Granger
##
##  Granger causality H0: DCOILWTICO do not Granger-cause diff_ppi
##
## data:  VAR object var_model
## F-Test = 1.8145, df1 = 1, df2 = 298, p-value = 0.179
##
##
## $Instant
##
##  H0: No instantaneous causality between: DCOILWTICO and diff_ppi
##
## data:  VAR object var_model
## Chi-squared = 63.393, df = 1, p-value = 1.665e-15
```

Create and graph a forecast of your assigned variable.

```r
forecast_var = predict(var_model, n.ahead = 4, ci = 0.95)
forecast_mean = forecast_var$fcst$DCOILWTICO[, "fcst"]
forecast_lower = forecast_var$fcst$DCOILWTICO[, "lower"]
forecast_upper = forecast_var$fcst$DCOILWTICO[, "upper"]
forecast_df = tibble(
  Time = (nrow(var_data) + 1):(nrow(var_data) + 4),
  Forecast = forecast_mean,
  Lower95 = forecast_lower,
  Upper95 = forecast_upper
)
actual_df = var_data %>%
  mutate(Time = 1:nrow(var_data)) %>%
  dplyr::select(Time, DCOILWTICO) %>%
  rename(Value = DCOILWTICO) %>%
  mutate(Type = "Actual")
forecast_plot_df = forecast_df %>%
  rename(Value = Forecast) %>%
  mutate(Type = "Forecast")
full_plot_df = bind_rows(actual_df, forecast_plot_df)
library(ggplot2)

full_plot_df %>%
  ggplot(aes(x = Time, y = Value, color = Type)) +
  geom_line() +
  geom_ribbon(data = forecast_df, aes(x = Time, ymin = Lower95, ymax = Upper95),
              inherit.aes = FALSE, alpha = 0.2) +
  labs(title = "Crude Oil Price Forecast (VAR(1))",
       x = "Time", y = "Price (DCOILWTICO)") +
  theme_minimal()
```



Comment on whether this forecast is better or worse than the univariate forecast from question 1.

Comparing the forecasts from the univariate ARMA(1,1) model we computed in Question 1, and the multivariate VAR(1) model in Question 2, we observe that the forecast looks broadly similar in terms of forecast level and volatility and no visible changes are noticed. The Granger Causality tests earlier showed only weak evidence that diff_ppi improves crude oil price forecasts, the additional information from the VAR model is unlikely to offer substantial gains in forecast accuracy over our initial approach. Thus, we can conclude by saying that the univariate ARMA(1,1) model appears to perform as well as the VAR(1) model for forecasting crude oil prices, if not slightly better.